

Three Roads: Choose Your Stack with AI

1. My Constraints

I used four real constraints to choose the stack rather than asking AI to simply pick a framework for me.

Free only: I need the portfolio to be built and hosted without requiring paid hosting or paid services.

My skill level: I have practical experience with modern web development and have worked with React, Next.js, TypeScript, and other development tools, but I am still developing my skills and would not describe myself as an expert in production web architecture.

What my portfolio needs to do: My sitemap is organized around three main areas: Home, Work, and About/Contact. Home introduces me and previews my work; Work contains security write-ups including XSS and SQL injection case studies; About/Contact provides background, an email CTA, and links to GitHub, LinkedIn, and my resume. The case studies need to support a clear structure such as Scope, Method, Findings, and Impact.

How my work must be displayed: The portfolio needs to display security evidence and screenshots, image-based work, long-form case-study reading, code repository links, and professional links. A live embedded demo is not currently required.

Dynamic functionality: Nothing needs to be dynamic yet. The current portfolio is mainly a content and presentation site, so I do not currently need a database, authentication, or custom backend.

2. Three Stack Options

Option 1 — Simplest: HTML + CSS + JavaScript

How I would build it: I would create the pages directly with HTML, style them with CSS, and use JavaScript only where small interactions are necessary. The images and case-study content would live in the repository.

Free hosting: GitHub Pages. GitHub Pages is available with GitHub Free for public repositories and is designed to publish HTML, CSS, and JavaScript from a repository.

Backend: No. The current portfolio does not need one.

Real trade-off: This is the easiest option to understand and maintain at a basic level, but repeated page structures and case-study layouts would become more manual as the portfolio grows. I would gain simplicity but give up some component reuse and scalability.

Option 2 — Middle: Astro

How I would build it: I would use Astro for reusable components and static pages, with the case-study content organized so that additional projects could be added without manually duplicating the entire page structure.

Free hosting: Cloudflare Pages.

Backend: No. The current portfolio can remain a static site.

Real trade-off: Astro gives me a cleaner structure for a content-heavy portfolio than plain HTML, while staying relatively lightweight. However, it introduces another framework that I would need to learn and maintain. It is a good middle ground, but I would be adopting a new stack even though I already have experience with Next.js.

Option 3 — Most Powerful: Next.js + TypeScript

How I would build it: I would build the portfolio as a Next.js application using TypeScript and reusable components. The current pages and case studies could be statically generated, while images, long-form content, and project sections could be organized into reusable structures.

Free hosting: Vercel Hobby. Vercel currently offers a free Hobby plan, with the important limitation that its terms specify personal/non-commercial use. That matches the intended use of this personal portfolio.

Backend: Not yet. I would start without a database or custom backend because the current requirements do not justify one. Next.js gives me the option to add server-side functionality later if a real requirement appears.

Real trade-off: This is more complex than the other two options. I would have more dependencies, framework conventions, and TypeScript/React concepts to maintain. The benefit is a reusable component architecture, a stack I already have practical experience with, and room to expand without having to migrate the portfolio to another framework later.

3. Pressure-Test of the Front-Runner

I chose Next.js + TypeScript as the front-runner, so I tested whether its extra power is actually justified.

What breaks if I choose the simplest option?

Nothing important breaks immediately. HTML, CSS, and JavaScript can display all of my current work, including screenshots, case studies, links, and long-form writing.

The problem is maintainability as the site grows. Repeated layouts and case-study structures would become more manual. If I later wanted a more structured component system, I could end up restructuring or migrating the project.

What do I maintain if I choose the most powerful option?

I would maintain a more sophisticated development environment, including Next.js, React, TypeScript, dependencies, and the framework's conventions. This is more work than plain HTML.

However, I already have practical experience with this ecosystem, so the learning and maintenance cost is reasonable for me. I am not choosing Next.js because I need every feature it provides; I am choosing it because the additional structure and flexibility are useful without requiring me to activate a backend now.

Can I finish in two weeks?

Yes. The actual site structure is relatively small: Home, Work, About/Contact, and the security case studies. I would not need to build a database, authentication system, or other complex application features. I can keep the first version focused on the actual portfolio requirements.

Does it show my work the way it needs to be shown?

Yes. Next.js can handle the required screenshots and image evidence, long-form case studies, structured project sections, repository links, resume links, and contact information. It also gives me reusable components for consistent case-study layouts.

The important point is that the portfolio does not need to become a dynamic application just because I chose Next.js.

4. My Decision

I chose **Next.js + TypeScript, hosted on Vercel Hobby**.

I considered plain HTML/CSS/JavaScript and Astro as genuine alternatives. HTML/CSS/JavaScript was the simplest and would work for the current requirements, but I would have more manual maintenance as the site grows. Astro was a strong middle option because it provides a lightweight structure for content-heavy sites, but it would mean learning and maintaining another framework when I already have practical experience with Next.js.

I chose Next.js because it gives me a good balance between what I need now and where the portfolio could go later. I can keep the current site static and simple while still having a component-based architecture and the ability to expand the project without immediately changing stacks.

Can I maintain this? Yes. It is more complex than plain HTML, but it is within my current skill set and I already have experience with the Next.js/TypeScript ecosystem.

Does it show my work well? Yes. It supports the screenshots, security evidence, long-form case studies, code repositories, resume, and professional links that the portfolio needs.

Does it need a backend now? No. The current requirements are content-focused, so I will not add a backend until there is a real requirement for server-side functionality, such as database-backed content, authentication, an API, or another genuinely dynamic feature.

My decision is therefore to use **Next.js + TypeScript + Vercel Hobby**, starting as a static portfolio and adding backend functionality only if the portfolio later develops a requirement that actually needs it.